

Is Dijkstra's algorithm for directed or undirected graphs?

Asked 9 years, 4 months ago Modified 3 years, 7 months ago Viewed 73k times



40

I keep trying to google this, but the results I'm finding are just adding to my confusion. It seems that it can be possibly used for both? If so, which is it designed for by default and what needs to change to make it work the non-default way (whether that be directed or undirected)?



Edit: for reference, I had a problem last semester where I was given a list like this (airports):



AER, KZN, 1.8835
ASF, KZN, 1.3005
ASF, MRV, 1.1204
CEK, KZN, 1.9263
CEK, OVB, 1.6733
DME, KZN, 1.7892
DME, NBC, 2.2319
DME, UUA, 2.3786
EGO, KGD, 1.4649
EGO, KZN, 1.2603
GYD, NBC, 2.0755

I was told it was directed, and asked to find the shortest path. I put it into a Dijkstra's algorithm I found on Github (it was an open-computer midterm so we didn't have nearly enough time to write the algorithm from scratch) and my professor said the shortest path it returned was incorrect and that it was not even a possible path because the list was supposed to be directed. I wasn't sure if I was supposed to then modify the algorithm or the list to make this correction. It ended up being the case that the 2nd shortest path it returned was actually the directed shortest path, but I'm still wondering what the problem was.

algorithm

graph

dijkstra

Share Improve this question Follow

edited Jul 4, 2016 at 19:20

asked Jul 4, 2016 at 18:37



Austin

7,419

13

80

154

6 Answers

Sorted by: Highest score (default)



67

It can be applied to both. Here is why:

An *undirected* graph is basically the same as a *directed* graph with **bidirectional** connections (= two connections in opposite directions) between the connected nodes.



So you don't really have to do anything to make it work for an undirected graph. You only need to know all of the nodes that can be reached from every given node through e.g. an **adjacency list**.

Share Improve this answer Follow

answered Jul 4, 2016 at 18:49



Keiwan

8,311 5 38 49

! Sign up to request clarification or add additional context in comments.



6 Comments

Add a comment



Austin Over a year ago

So if my list doesn't include a duplicate but reversed entry for each edge it should give me the directed shortest path? Because I was told the list I was given was directed, but the implementation of Dijkstra's I used gave me an undirected shortest path.

▲ 0

Reply



Keiwan Over a year ago

Well, this sounds like the implementation you found interprets the entries in your list as undirected, which is not what you want so you should probably just implement the algorithm yourself. The ones I know generally just use an adjacency list (which is a list that contains information about which nodes you can reach from a given node => *the connections are directed*). This is why I said you don't have to do anything to make it work for undirected graphs, however, if your algorithm for whatever reason automatically interprets the connections as bidirectional it needs to be changed.

▲ 2

Reply



WSS Over a year ago

@Keiwan This post [stackoverflow.com/questions/22649416/...](https://stackoverflow.com/questions/22649416/) says prims algorithm will fail for directed graphs. So Dijkstras which is almost like prims will also fail for examples given in the post.

▲ 0

Reply



Keiwan Over a year ago

But Dijkstra and Prim/Kruskal calculate two inherently different things. Dijkstra calculates the shortest path tree from *one* node whereas Prim/Kruskal calculates the minimum spanning tree between *all* nodes. The concept of an MST is not defined for directed graphs - the connections have to be undirected. For a directed graph you'll be looking to find a minimum cost aborescence, which can't be done using Prim/Kruskal. This has nothing to do with Dijkstra though. The concept of the shortest path tree is well defined for directed graphs, which is why Dijkstra also works on them.

▲ 2

Reply



WSS Over a year ago

But does not applying Dijkstra for vertex s on second example gives wrong answer ?

▲ 0

Reply





9



You can use Dijkstra's algorithm in both directed and undirected graphs, because you simply add nodes into the PriorityQueue when you have an edge to travel to from your adjacency list. For example, if one of my nodes has an edge from A to B of weight 3, then if it's directed from B I won't be able to add the edge back into A, while from A I could add it to B.

Like the other answers, make sure you do not confuse it with weights. Dijkstra's algorithm runs on positive weighed graphs, otherwise the priority queue would be useless.

In your example, Dijkstra's algorithm would work because the graph is both weighed (positively) and has directed edges.

The fault would have been that the edges have been double-assigned in the form of an undirected graph. You should be careful when parsing the edges at the beginning into your objects to not duplicate the edges in the adjacency list.

Share Improve this answer Follow

edited Apr 1, 2022 at 16:49



Ryan M ♦

20.6k 35 74 85

answered Jul 7, 2016 at 14:26



MathBunny

956 1 9 17

Comments

Add a comment



7



Dijkstras algorithm is typically for **Positive** weighted graphs. Perhaps you are confusing it with the breadth first search (BFS) algorithm, which is essentially Djikstras for unweighted graphs. The difference (between Djikstras and BFS), is when you are dealing with weighted paths, we must now consider the path costs adjustments (weights) & the decisions on which nodes to visit after the current.

Share Improve this answer Follow

answered Jul 4, 2016 at 18:57



Luvy

96 2

1 Comment ▾

Add a comment



bit_cracker007 Over a year ago

It works for negative weighted graphs too (except for negative cycles, where it can end up optimizing forever).

▲ 0

Reply





5

A graph being directed just means that the edges connecting vertices are able to connect one way, but not the other. This means that one vertex can be adjacent to another, but that other vertex may not be adjacent to the first vertex.



In the context of Dijkstra's algorithm, whether the graph is directed or undirected does not matter. Dijkstra's algorithm simply references the adjacent vertices of a vertex. It is this adjacency list that you would have to modify if you were changing a graph from directed to undirected.



Share Improve this answer Follow

answered Jul 4, 2016 at 18:50



[Aaron Wojnowski](#)

6,490 5 31 50

6 Comments

Add a comment



[Austin](#) Over a year ago

I ask because I pulled an implementation of Dijkstra's off Github to find the shortest path in a list of airports, where my instructor said the list was supposed to be directed, but the shortest path it returned ended up being the undirected shortest path. So in this case I would have to change the list rather than the algorithm?



0



Reply



[Aaron Wojnowski](#) Over a year ago

You have to make sure that the airport list contains direction information and that your algorithm takes note of the directions in the graph. Figure out which one of those conditions (or both) isn't satisfied and adjust accordingly. It would probably be more beneficial for you to implement the algorithm yourself though, since it sounds like it's for academic work. Referencing is okay, but get out of the habit of rote copying.



1



Reply



[Austin](#) Over a year ago

So for instance, should it see `AER, KZN, 1.8835` as a directed edge and require `KZN, AER, 1.8835` in addition to work bidirectionally? Because I don't believe my list had both entries, but it still considered the reverse orders as edges. Is this normal?



0



Reply



[Aaron Wojnowski](#) Over a year ago

Requiring both entries for a bidirectional graph and seeing one entry as directed would make sense, in my opinion. It sounds like your implementation of Dijkstra's algorithm was treating one entry as bidirectional.



1



Reply



[Austin](#) Over a year ago

Ok thanks, starting to understand I think. So in this case I would have needed to look through the algorithm's implementation and remove code where it also adds the reverse's to the adjacency list?



0



Reply





1

Dijkstra can work for Cyclic graphs, too. Which implies it can work for Undirected graphs. Because an Undirected edge in a graph can simply be assumed to be bidirectional edge. A to B. And B to A. Dijkstra's works for this.



Share Improve this answer Follow

edited Apr 1, 2022 at 16:49



Ryan M ♦

20.6k 35 74 85

answered Jul 21, 2021 at 17:48



Aditya Singh

127 8



Comments

Add a comment



0

Yes Dijkstra work for both directed & undirected graph but all edge weight should be +ve . Because if any weight is -ve, then it may fail to give the correct answer. It works on undirected graph because in Dijkstra, we should always seen that minimum edge weight. From its source vertex



Share Improve this answer Follow

edited Apr 1, 2022 at 16:51



Ryan M ♦

20.6k 35 74 85

answered Aug 21, 2019 at 5:51



Gulshan Kumar

1



Comments

Add a comment

Start asking to get answers

Find the answer to your question by asking.

Ask question

Explore related questions

algorithm

graph

dijkstra

See similar questions with these tags.